Scroll nets

Towards a structural proof theory of Peirce's existential graphs

Pablo Donato
Charles University, Czechia

Abstract

We introduce a new formalism for representing proofs in propositional logic called "scroll nets". Its fundamental construct is the scroll, a topological notation for implication proposed by C. S. Peirce at the end of the 19th century as the basis for his diagrammatic system of existential graphs (EGs). Scroll nets are derived from EGs by following the Curry-Howard methodology of internalizing inference rules inside judgments, just as terms in type theory internalize natural deduction rules. We focus on the intuitionistic implicative fragment of EGs, starting from an intuitive diagrammatic notation for scroll nets, and then distilling their combinatorial essence into an inductive definition. We also identify a notion of detour, as well as an equational theory inspired by cartesian monoidal categories that can be oriented to perform detour elimination. We illustrate how to simulate normalization in the simply typed λ-calculus, demonstrating both the logical and computational expressivity of our framework.

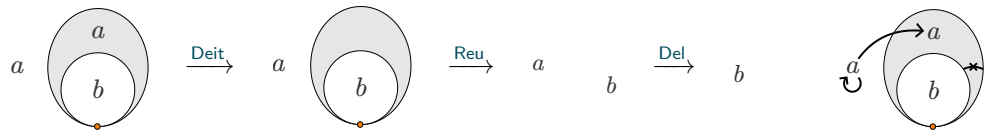
2012 ACM Subject Classification Theory of computation → Proof theory; Theory of computation → Type theory; Theory of computation → Equational logic and rewriting; Theory of computation → Interactive computation

Keywords and phrases deep inference, graphical calculi, existential graphs, Curry-Howard correspondence, simply-typed lambda-calculus

1 Introduction

Graphical proof theory Traditionally, proof theory has been established as a branch of *symbolic* logic: it embodies *par excellence* the Hilbertian ideal according to which any mathematical concept can be expressed as a finite sequence of symbols, with reasoning reduced to the sequential manipulation of these symbols. However, several recent developments tend to demonstrate that a finer analysis of the structure of formal proofs is possible with the help of *graphical* or diagrammatic representations: popular examples include Girard's proof nets in linear logic [6], string diagrams in categorical logic [18], and Hughes' combinatorial proofs [15]. The idea is to abstract from the arbitrary sequentiality suggested by a linguistic usage of symbols, by immersing statements and proofs in space to better understand their geometry. One aim is to find a solution to Hilbert's 24th problem by devising a good notion of *equality* among proofs [30]. However, current research focuses mostly on the structure of *completed* proofs, forgetting the sequential inference process that enabled their construction.

Existential graphs To tackle this limitation, we investigate a graphical system mostly forgotten by contemporary proof theorists, probably because it predates the existence of proof theory itself: C. S. Peirce's *existential graphs* [26] ("*EGs*" hereafter). It is based on a purely diagrammatic and topological representation of logical constants, inspired by a dialogical understanding of reasoning a century before the advent of game semantics [25]. Proving is then modelled as a dynamic process of deriving valid statements through rewriting of diagrams, using six elementary inference rules that perform insertions and deletions of graphs at a single specified location. Figure 1a illustrates how to derive *modus ponens* in EGs, starting from a graph of the statement $a \wedge (a \Rightarrow b)$ and reducing it to b.



(a) Dynamic proof trace **(b)** Static proof object

Figure 1 Modus ponens in scroll nets

	Proof trace	Proof object
Gentzen calculi	Sequent calculus derivation (*tree*)	Natural deduction derivation (*tree*) Proof net (*graph*)
Deep inference	Calculus of structures (*sequence*) Open deduction (*sequence/tree*)	Atomic flow (*graph*) Combinatorial proof (*graph*)
Rocq/Lean	Proof script (= tactic *sequence*)	Proof term (= natural deduction *tree*)
String diagrams	Equational rewriting (*sequence*)	—
Existential graphs	Illative transformations (*sequence*)	Scroll net (*bigraph*)

Table 1 Comparing representations of proofs in various formalisms

What is a proof? We believe that any approach to (structural) proof theory should account for two complementary aspects of a proof: the *dynamic* inference process that constructs it, which we call a *proof trace* (traditionally called "derivation"), and the *static proof object* that it witnesses. This distinction has a long history: the sequent calculus was originally considered by Gentzen as a meta-calculus for constructing natural deduction proofs [7], the latter being the "real" proof objects. Girard later extended this reading to linear logic by introducing proof nets as its "natural deduction" [6]. The same dichotomy underlies the Curry-Howard correspondence and the interface of state-of-the-art ITPs such as Rocq [31] and Lean [20], where users write a *proof script* to progressively elaborate a *proof term*. Table 1 summarizes how various graphical and symbolic formalisms instantiate this distinction. At one extreme, string diagrams and Peirce's original EGs offer only a proof trace — a sequence of equational rewritings — with no explicit counterpart proof object. At the other extreme, even when both are present, a single proof object typically witnesses proof traces that disagree on more than the mere order of independent inferences, so that some of the trace's fine structure is lost in the object.

Scroll nets The aim of the current paper is to introduce a language of *proofs* on top of the existing language of statements in EGs, in order to represent proof objects in addition to proof traces. A notable feature of EGs is that they identify the language of statements with the language of *judgments*: scribing some graph on the *sheet of assertion* is, by definition, the same as asserting its truth[1]. Following the Curry-Howard methodology, our goal is then to internalize inference rules — Peirce called them *illative transformations* — inside the syntax of statements, just like terms in type theory can be seen as a way to internalize derivation trees inside judgments.

The syntax we have arrived at is based on the simple observation that all illative transformations can be expressed in terms of pure *locations*: they consist in either *inserting/deleting* an

[1] This identification is also found in the original notation of Gentzen for natural deduction, where nodes in derivation trees are formulas rather than sequents [7].

arbitrary graph at a given location, or *(de)duplicating* an arbitrary graph from a source/target location. This gives rise to a directed forest whose nodes are the "subgraphs" of some EG and whose edges encode exactly illative transformations. This data structure is surprisingly reminiscent of the *proof nets* of linear logic, and because it is based on the fundamental construct of the *scroll* coming from intuitionistic EGs, we call it ***scroll net***. Figure 1b shows an example of scroll net, which is exactly the proof object built by the derivation of Figure 1a. Notably, all proof traces yielding a given scroll net consist of the same illative transformations, differing only in their order of execution.

Outline The article is organized as follows: in Section 2 we recall the diagrammatic syntax of the implication-conjunction fragment of intuitionistic EGs, based on the fundamental icon of the scroll. In Section 3 we explain how to internalize illative transformations inside EGs by using a diagrammatic arrow notation. In Section 4 we give a formal definition of scroll nets based on an inductive grammar. In Section 5 we give a sequential correctness criterion for scroll nets following the dynamic understanding of illative transformations. In Section 6 we identify four kinds of detours that arise when a node is both introduced and eliminated. We also give a direct translation of simply typed λ-calculus into scroll nets, illustrating how to simulate β-reduction. We conclude in Section 7 with a discussion of related works and future directions to improve our framework and extend it to richer logics.

2 Implicative existential graphs

Sheet of Assertion The most fundamental concept of EGs is the *sheet of assertion*, denoted by SA thereafter. It is the space where statements are scribed by the reasoner, typically a sheet of paper, a blackboard, or a computer display. This last analogy suggests an important property of SA: it must offer a *virtually infinite* amount of space, so that one can perform as much reasoning as needed by scribing an unbounded (but finite) amount of statements[2].

Then as the name indicates, scribing some statement Φ on SA has the meaning of *asserting the truth of* Φ. It is thus an instance of the notion of *judgment* as identified by logicians like Frege and Martin-Löf, who would write it symbolically as $\vdash \Phi$. Naturally, the empty SA in interpreted as the absence of any assertion/judgment, and thus as vacuous truth $\top$.

Atoms Peirce often used sentences expressed in natural language as the most elementary statements scribed on SA, probably for pedagogical purposes. However he made clear that the informal meaning of these sentences, that is their denotation in the real world, is irrelevant to the process of pure logical deduction. This agrees with the modern view on *atomic* propositions, which are taken to be arbitrary abstract symbols drawn from some countably infinite set $\mathcal{A}$. We will use letters $a, b, c \ldots$ to denote such statements and call them *atoms*.

Juxtaposition Recall that one can scribe an arbitrary number of statements on SA, thus asserting the truth of each of them simultaneously. That is, *juxtaposition* has the meaning of *conjunction*, as we know from the introduction rule for $\wedge$ in natural deduction. However, symbolic connectives do not exist in the syntax of EGs, because Peirce aimed precisely for a

[2] Just like a Turing machine has an infinite tape, so that one can perform as much computation as needed. In symbolic logic, this is captured by the fact that formulas, although usually finite, can have an unbounded size.

symbolless — what he called *iconic* [28] — notation for logic. Thus very concisely,

$$\frac{\vdash \Phi \quad \vdash \Psi}{\vdash \Phi \wedge \Psi}\wedge\mathsf{i} \qquad \text{is expressed by the graph} \qquad \Phi \quad \Psi$$

Note that in symbolic logic, Φ and Ψ can be arbitrarily complex formulas, not just atoms. In EGs, a complex statement — that we will call a *graph* — is any delimited portion/area of SA, as long as the delimitation does not interrupt the continuity of some *token*.

Scroll In the fragment of EGs considered in this article, a token[3] is a scribed occurrence of either an atom, or what Peirce called a *scroll*. In the words of Peirce himself [24, pp. 533–534]:

> Accordingly, since logic has primarily in view argument, and since the conclusiveness of an argument can never be weakened by adding to the premisses or by subtracting from the conclusion, I thought I ought to take the general form of argument as the basal form of composition of signs in my diagrammatization; and this necessarily took the form of a "scroll", that is [...] a curved line without contrary flexure and returning into itself after once crossing itself, and thus forming an outer and an inner "close".

Examples of scrolls can be found in Figure 1, where the curved line is depicted by a white ellipse (the "inner close") nested in a gray ellipse (the "outer close"), with a unique orange intersection point emphasizing the "once crossing itself" part. Peirce also used the term *sep* instead of "close" to refer to any of the two closed curves that make up a scroll[4]. Following Pietarinen [17], we will use the terms *inloop* and *outloop* to designate the inner and outer seps of a scroll, respectively. By "general form of argument", Peirce alludes to logical *implication* $\Phi \Rightarrow \Psi$, where the antecedant Φ resides in the outloop, and the consequent Ψ in the inloop. Φ and Ψ can be arbitrarily complex graphs, so that in particular a scroll located in the outloop of another scroll will be drawn with its gray and white shading inverted in order to reflect the change of polarity (see for instance Figure 3a).

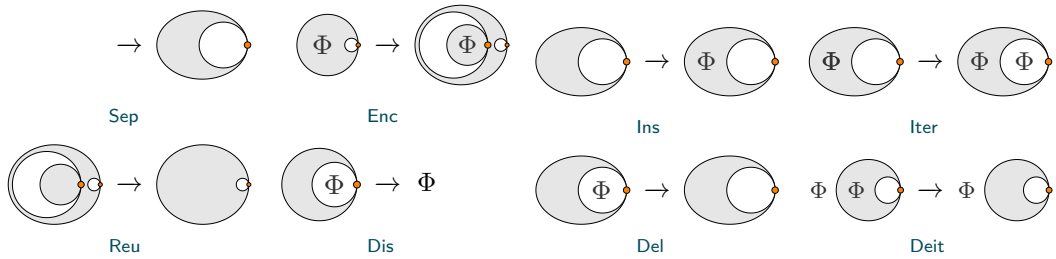
Finally, note that in classical logic seps can be interpreted as *negations* because of the classical equivalence $\Phi \Rightarrow \Psi \simeq \neg(\Phi \wedge \neg\Psi)$. Indeed EGs were invented around 1896 before the advent of intuitionistic logic, so that Peirce had a boolean interpretation of logical operations in mind. However in this article, we will demonstrate that the inference rules of EGs are not only intuitionistically sound (when restricted to scrolls), but that the notion of computation that emerges from them is very much in line with the modern conception of constructivism.

Illative transformations Peirce called the inference rules of EGs *illative transformations*. They are traditionally understood as *rewriting rules*, similarly to the rules of string diagram calculi in category theory [3], or to those of the *calculus of structures* in deep inference [10]. One can find various presentations in the literature, some of the rules being *equational* in nature (bidirectional) because of the equivalence between premiss and conclusion, the others being necessarily *oriented* (unidirectional). Here we stick to a fully oriented presentation, for reasons that will become clear in the next section. This makes for a total of eight rules, illustrated with representative examples in Figure 2. They should be read from left to right as expressing *forward reasoning* from premiss to conclusion.

---

[3] Peirce used the word "token" as a synonym of the more contemporary term "occurrence", and the word "type" to refer to the common pattern that is instantiated in multiple occurrences of the same type.

[4] This is because a closed curve literally *sep*arates SA into two distinct areas, which is now known as the *Jordan curve theorem* in topology.



(a) Interaction rules **(b)** Argumentation rules

Figure 2 Dynamic illative transformations

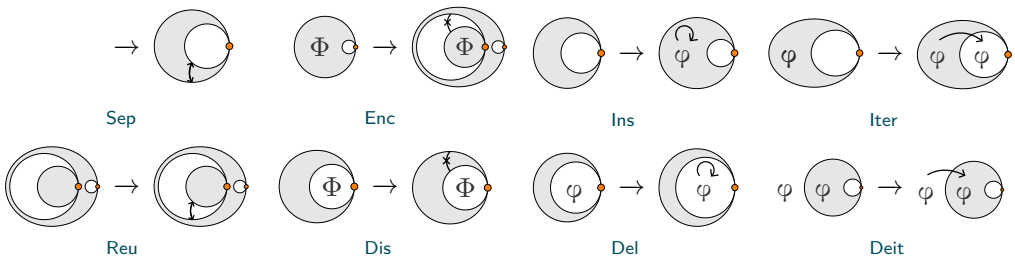
While still informal, the following description specifies the rules in their full generality. Note that to avoid any ambiguity — and to foster the parallel with natural deduction — we will use the terms *introduction* and *elimination* to refer respectively to the act of *scribing* and *erasing* a graph from SA. We will also refer to white/gray-shaded areas — or equivalently to areas enclosed in an even/odd number of seps — as *positive*/*negative* locations.

Separation **(Sep)** An empty scroll can be introduced in a positive location.
Reunion **(Reu)** An empty scroll can be eliminated from a negative location.
Enclosure **(Enc)** A scroll with empty outloop can be introduced around any graph Φ in a negative location.
Disclosure **(Dis)** A scroll with empty outloop can be eliminated around any graph Φ in a positive location.
Insertion **(Ins)** Any graph Φ can be introduced in a negative location.
Deletion **(Del)** Any graph Φ can be eliminated from a positive location.
Iteration **(Iter)** Any graph Φ can be introduced in a positive location, as long as Φ already occurs in the area of a sep that contains said location.
Deiteration **(Deit)** Any graph Φ can be eliminated from a negative location, as long as Φ already occurs in the area of a sep that contains said location.

A remarkable feat of Peirce's rules — on which he insisted very much — is that they are only expressed in terms of introductions (first row) and eliminations (second row) of graphs on SA. We will refer to this property as *illative atomicity*. Indeed, Peirce thought that those were the *smallest* steps in which reasoning could be dissected, making his system extremely appropriate for *analytical* purposes. This is summarized in the following excerpt [24, p. 533]:

> In the first place, the most perfectly analytical system of representing propositions must enable us to separate illative transformations into indecomposable parts. Hence, an illative transformation from any proposition, A, to any other, B, must in such a system consist in first transforming A into AB, followed by the transformation of AB into B. For an omission and an insertion appear to be indecomposable transformations and the only indecomposable transformations.

We qualify the first four rules Sep, Reu, Enc, Dis as *interaction* rules. Indeed from a game-semantical point of view, the Sep (resp. Enc) rule starts an interaction with the opponent by introducing a negative (resp. positive) outloop in a positive (resp. negative) area, while the Reu and Dis rules end this interaction by erasing the scroll while keeping the inloop's content. They are obviously sound in both intuitionistic and classical logic, as they correspond to the equivalence $\top \Rightarrow \Phi \simeq \Phi$.



(a) Interaction rules **(b)** Argumentation rules

Figure 3 Static illative transformations

We qualify the other rules as *argumentation* rules, following Peirce's description of the scroll as the "general form of argument". They are a strict generalization of the *structural rules* found in sequent calculus and the calculus of structures (as well as the *switch* rule in the latter), because they can be applied in locations of arbitrary depth and polarity, not just at the top-level of sequents or in positive contexts.

The Ins and Del rules correspond respectively to *weakening* and *coweakening*. They capture the monotonicity of entailment, expressed by Peirce in the sentence "[...] the conclusiveness of an argument can never be weakened by adding to the premisses [(Ins rule)] or by subtracting from the conclusion [(Del rule)] [...]".

Lastly, the (de)iteration rules Iter and Deit can be seen respectively as generalizations of the *cocontraction* and *contraction* rules. They capture when some source occurrence of a statement Φ *justifies* a distinct target occurrence of the same Φ, where *justification* consists in either adding a positive conclusion (Iter) or removing a negative assumption (Deit). Their soundness is less straightforward than for other rules, and for lack of space we rely on secondary sources like [5] where it has been proved formally with respect to intuitionistic Kripke semantics, albeit in a slightly different setting.

3 Reifying inference

Justifications In all presentations of illative transformations found in secondary literature, iteration and deiteration are *polarity-independent*: they can each justify statements in both positive and negative locations. Then deiteration can be seen as the converse of iteration, and since the premiss and conclusion are equivalent, they are usually presented as a single equational rule. However, Peirce himself noted that this presentation is "[...] valid, sufficient for its purpose, and convenient in practice, [...][but there is a] more scientific way [to proceed]" ([24, p. 536]). Then he goes on to explain this "more scientific way", of which one clause reads as follows [24, p. 537]:

> [...] if Ω be a **recto [(i.e. positive)]** area, any simple Graph already scribed upon Y may be iterated upon Ω; while if Ω be a **verso [(i.e. negative)]** Area, any simple Graph already scribed upon Y and iterated upon Ω may be deiterated by being deleted or abolished from Ω.

This subtle polarity restriction turns out to be very important in order to *reify* (de)iteration in the syntax of EGs. Indeed, recall that our goal is to find a way to *record* illative transformations on SA as they are being performed, so that the result of a sequence of such transformations is the static proof object that was built, rather than its mere conclusion;

just as the conclusion of a derivation in type theory is a judgment that holds not only a type, but also a term witnessing this type.

Then, since a (de)iteration justifies a target occurrence of some graph Φ with a source occurrence of Φ, it is tempting to represent it by a simple *arrow* with corresponding source and target. More precisely, one can implement (de)iteration of an arbitrary graph $\Phi = \phi_1, \ldots, \phi_n$ as n parallel arrows on the copies of its tokens ϕ_i, representing n applications of the rule Iter or Deit[5]. This gives the *static* versions of Iter and Deit in Figure 3b. Note that since we want all transformations to strictly *add* information incrementally, the justified occurrence of ϕ in Deit is not erased anymore. Then the only way to distinguish between these static versions of Iter and Deit is to look at the polarity of the target location.

The same principle can be applied to the Ins and Del rules, which are now represented by *looping arrows*. The intuition is that a token introduced in a negative area with the Ins rule corresponds to an *assumption*, which is *self-justified* in the sense that the reasoner decides it does not require further justification (avoiding the infamous infinite regression problem). Dually, a token eliminated from a positive area with the Del rule *annihilates itself*, capturing the reasoner's intent to prevent further usage of the knowledge about this token's truth.

Interactions There is a certain *homeomorphic* flavor to interaction rules, expressed by Peirce for the Dis rule as follows: "[...] the two walls [(i.e. seps)] of the scroll, when nothing is between them, fall together, **collapse**, disappear, and leave only the contents of the inner close standing [...]" ([24, p. 534]). We diagrammatize this "collapse" deformation by drawing two arrows departing from each sep and meeting at their tips, which also gives the impression of a *cross* symbolizing elimination (rules Reu and Dis) in Figure 3a). Dually, introduction of a scroll (rules Sep and Enc) is diagrammatized with a double-ended arrow whose tips touch the two seps, evoking an *expansion* movement as if SA was teared apart to create negative space. In order to stay consistent with the polarity-dependent interpretation of arrows in argumentation rules, collapse (resp. expansion) arrows are interpreted as introductions (resp. eliminations) in negative areas.

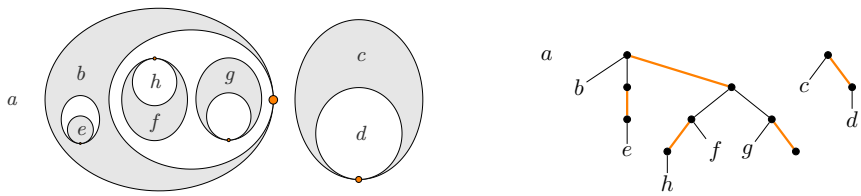
4 Inductive syntax

Scroll structures All the ingredients are now in place to define formally our notion of scroll net. However, as intuitive as the graphical notations introduced above can be for concrete manipulation purposes, the amount of space they require is not very well-suited to abstract meta-theoretical analysis. In a foundational article that introduced the intuitionistic interpretation of the scroll, Oostra observes that Peirce had already proposed a more concise *linear* and *symbolic* notation for it, where the outer and inner seps are written respectively with square brackets '[]' and parentheses '()' [22, p. 37].

Moreover as noted by various authors[6], the nesting of seps on SA induces a *tree* structure on EGs: each sep constitutes a node, whose children are either leaves corresponding to atoms residing in the area of the sep, or other nodes corresponding to nested seps. Empty seps have no children, and thus also form leaves of the tree. This is illustrated graphically in Figure 4b. Putting it all together, we get the following concise definition:

[5] Although these transformations are typically performed sequentially by a single reasoner (as will be modelled by our notion of derivation in Section 5), their relative order does not matter, so that one could very well imagine a completely parallel execution.

[6] See for instance the Tree Existential Graphs of Roberts and Pronovost [27], or [4, Section 2.2].



(a) Topological **(b)** Combinatorial

$$a, [b, [(e)]\,([f\,(h)]\,, [g\,()])]\,, [c\,(d)]$$

(c) Textual

Figure 4 Topological, combinatorial and textual representations of a scroll structure. In Fig. 4b, labelled leaves are atoms, •-nodes are seps, and orange edges are *attachments* of inloops to outloops.

▶ **Definition 1** (Scroll structure)**.** Simple scroll tokens ϕ *and* simple scroll structures Φ *are defined mutually inductively by the following grammar:*

$$\phi, \psi, \xi ::= a \mid [\Phi\,(\Psi)]$$
$$\Phi, \Psi, \Xi ::= \phi_1, \ldots, \phi_n$$

where $a \in \mathscr{A}$ *can be any atom.*

Figure 4c shows an example of simple scroll structure expressed in this textual syntax. As suggested by our choice of letters, Definition 1 captures Peirce's informal notions of graph and token. The adjective "simple" is here meant as a reference to *simple type theory*, as will become clear in Section 6. We will usually omit the qualificatives "simple" and "scroll" for conciseness.

For the correspondence between the topological and textual notations to be exact, it would seem that one needs to abstract from the particular order in which tokens occur in structures, i.e. the latter should be considered as (nested) *multisets* rather than *lists* of tokens. Indeed in the topological notation, what matters to distinguish two graphs is not the particular coordinates of tokens, but the areas delimited by seps to which they belong. This feature has been called the "*isotropy* of SA" in [17], and it allows to identify conjunctive statements modulo commutativity and associativity.

In sequent calculi, one standard way to formalize this is to literally define contexts Γ, Δ as multisets of formulas. However in his original formulation of sequent calculus, Gentzen preferred to define contexts as lists, with permutation of formulas reified as a structural rule of the system, the so-called *exchange* rule. One reason is supposedly the computational relevance of exchange in the cut-elimination process, where it specifies the order in which cuts are treated. For similar computational motivations, we will stick to the ordered list definition of scroll structures, invoking their equivalence up to permutation when required:

▶ **Definition 2** (Permutation equivalence)**.** *We say that two structures* $\Phi = \phi_1, \ldots, \phi_n$ *and* $\Psi = \psi_1, \ldots, \psi_n$ *are* permutation-equivalent*, written* $\Phi \equiv_p \Psi$*, if there exists a permutation* $\sigma : \{1, \ldots, n\} \to \{1, \ldots, n\}$ *such that for all* $1 \leq i \leq n$*, either* $\phi_i = \psi_{\sigma(i)} = a$*, or* $\phi_i = [\Phi_0\,(\Phi_1)]$ *and* $\psi_{\sigma(i)} = [\Psi_0\,(\Psi_1)]$ *with* $\Phi_0 \equiv_p \Psi_0$ *and* $\Phi_1 \equiv_p \Psi_1$*.*

Variables In the previous section, we saw that illative transformations can be internalized statically inside scroll structures, by simply adding various kinds of arrows between tokens.

In graphical notation, the source and target tokens of an arrow are immediately identifiable by virtue of the arrow's positioning. However in textual notation, one needs a more explicit mechanism to identify tokens *uniquely*, which can be achieved with the notion of *variable*. This gives us a slightly extended, *locative* notion of scroll structure that we call an *interface*:

▶ **Definition 3** (Interface). Simple scroll shapes X, simple scroll nodes N *and* simple scroll interfaces $\mathcal{X}$ *are defined mutually inductively by the following grammar:*

$$X, Y, Z ::= a \mid [\mathcal{X}\,(\mathcal{Y})] \qquad N, M ::= x : X$$
$$\mathcal{X}, \mathcal{Y}, \mathcal{Z} ::= N_1, \ldots, N_n$$

where we use letters $x, y, z \ldots$ to denote variables *drawn from some fixed denumerable set $\mathcal{V}$.*

In essence, an interface $\mathcal{X}$ is just a structure where all tokens are assigned a unique variable identifier. This is captured by the following *forgetful map*:

▶ **Definition 4** (Underlying structure). *The* underlying structure $\lfloor \mathcal{X} \rfloor$ *of an interface $\mathcal{X}$ is defined recursively as follows:*

$$\lfloor N_1, \ldots, N_n \rfloor = \lfloor N_1 \rfloor, \ldots, \lfloor N_n \rfloor \qquad \lfloor a \rfloor = a$$
$$\lfloor x : X \rfloor = \lfloor X \rfloor \qquad \lfloor [\mathcal{X}\,(\mathcal{Y})] \rfloor = [\lfloor \mathcal{X} \rfloor\,(\lfloor \mathcal{Y} \rfloor)]$$

For the rest of this paper we assume an analogue to Barendregt's convention, that *all nodes occurring in an interface must be identified by distinct variables, whatever their depth.* In particular, we assume that all operations manipulating interfaces will preserve this convention, which can easily be achieved through *renaming* with fresh variables:

▶ **Definition 5** (Renaming). *A* renaming *is a map $\rho : \mathcal{V} \to \mathcal{V}$. The application $\rho\mathcal{X}$ of a renaming ρ to an interface $\mathcal{X}$ is defined recursively as follows:*

$$\rho(N_1, \ldots, N_n) = \rho N_1, \ldots, \rho N_n \qquad \rho a = a$$
$$\rho(x : X) = \rho(x) : \rho X \qquad \rho\,[\mathcal{X}\,(\mathcal{Y})] = [\rho\mathcal{X}\,(\rho\mathcal{Y})]$$

Two interfaces $\mathcal{X}$ and $\mathcal{Y}$ are said to be α-equivalent*, written $\mathcal{X} \equiv_\alpha \mathcal{Y}$, when there exists a renaming ρ such that $\rho\mathcal{X} = \mathcal{Y}$. They are* p-equivalent *whenever their underlying structures are, i.e. $\mathcal{X} \equiv_p \mathcal{Y}$ iff $\lfloor \mathcal{X} \rfloor \equiv_p \lfloor \mathcal{Y} \rfloor$. Note that α-equivalence implies p-equivalence.*

Justifications A node in an interface can be the source of arbitrarily many justification arrows added by (de)iterations (Figure 3b), but the target of at most one: in the dynamic reading of these rules, a node is introduced or eliminated (or both) only once. We write y/x to denote that x is justified by y, a notation that evokes (explicit) substitution [2]. Each node x thus has a single potential source justification y/x, but may itself serve as source for many others $x/z_1, \ldots, x/z_n$.

In the graphical notation of Figure 3b, a node can also be self-justified through the Ins or Del rule by being equipped with a looping arrow. Here we will write $\dot{x}$ to denote that x is self-justified, which should be understood as a form of variable *binding*. More precisely, we will see in Section 6 that self-justified nodes in positive and negative locations play respectively the role of `let` *bindings* (or *explicit substitutions* [2]) and λ-*abstractions*.

Finally, a node x can be inserted in a negative area and then deiterated by y, or iterated by y in a positive area and then deleted. In both cases we will write it $y/\dot{x}$, combining the two previous notations.

Scroll nets In summary, any node x can be in one of four different justification *states*: *unjustified* x, *ext-justified* y/x (externally justified by y), *self-justified* $\dot{x}$, or both $y/\dot{x}$. Simarly, any scroll can be in one of four states with regard to interaction: *idle* $[()]$, *expanded* $[\guillsinglleft()]$ (Sep and Reu rules), *collapsed* $[()\guillsinglright]$ (Enc and Dis rules), or both $[\guillsinglleft()\guillsinglright]$. This gives us the last ingredient for our formal definition of *scroll nets*:

▶ **Definition 6** (Scroll net)**.** Scroll states S, judgments J *and* scroll nets $\mathfrak{S}$ *are defined mutually inductively by the following grammar:*

$$\begin{array}{llll} S,T,U ::= a \mid [\mathfrak{S}\,\langle\mathfrak{T}\rangle] & J,K,L ::= \mathfrak{j} : S & \langle ::= (\mid \guillsinglleft(\\ \mathfrak{S},\mathfrak{T},\mathfrak{U} ::= J_1,\ldots,J_n & \mathfrak{j},\mathfrak{k},\mathfrak{l} ::= x \mid \dot{x} \mid y/x \mid y/\dot{x} & \rangle ::=) \mid)\guillsinglright \end{array}$$

Note that interaction states are decomposed further into an *expansion* state (opening angle bracket '$\langle$') and a *collapse* state (closing angle bracket '$\rangle$'), in order to make later definitions by case analysis more concise. We also reuse the terminology of *judgment* from type theory with its colon notation $\mathfrak{j} : S$, where $\mathfrak{j}$ is the *justification* of the state S[7].

▶ Remark 7. Interfaces correspond to the (very small) subset of scroll nets where all states are both idle and unjustified.

Oftencase, a judgment in a scroll net is never used to justify other judgments through (de)iteration, so that there is no occurrence of its variable x used in some justification x/y. In such cases, we will omit by convention the judgment's variable, writing respectively S, $\cdot S$, z/S or $z/\cdot S$ instead of $x : S$, $\dot{x} : S$, $z/x : S$ or $z/\dot{x} : S$.

▶ **Example 8.** The scroll net of Figure 1b can be expressed in textual notation as

$$\dot{x} : a, [x/a\,(b)\guillsinglright]$$

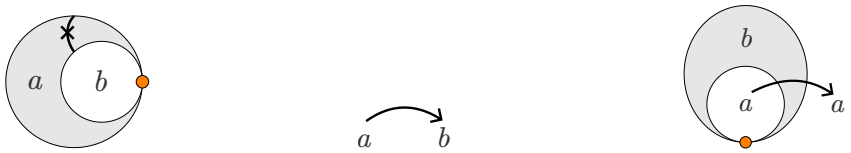
Boundaries Given a scroll net, it is not immediately obvious *what* it is proving — i.e. what are its *premiss* and *conclusion*, because both are "superposed" in a non-trivial way. For instance in the scroll net of Figure 1b, the same atom b occurs both in the premiss and the conclusion. This is actually a powerful feature, as it makes proofs very compact by identifying occurrences that would appear separate in most other proof formalisms[8], including proof nets and combinatorial proofs. Fortunately there is a simple algorithm for disentangling scroll nets, which exploits the illative atomicity and duality of inference rules:

▶ **Definition 9** (Boundaries)**.** *The* premiss $\lceil\mathfrak{S}\rceil$ *and* conclusion $\lfloor\mathfrak{S}\rfloor$ *of a scroll net* $\mathfrak{S}$ *are interfaces defined mutually recursively as follows:*

$$\begin{array}{rcl} \lfloor J_1,\ldots,J_n\rfloor = \lfloor J_1\rfloor,\ldots,\lfloor J_n\rfloor & & \lceil J_1,\ldots,J_n\rceil = \lceil J_1\rceil,\ldots,\lceil J_n\rceil \\ \lfloor \dot{x} : S\rfloor = \lfloor y/\dot{x} : S\rfloor = \square & & \lceil y/x : S\rceil = \lceil y/\dot{x} : S\rceil = \square \\ \lfloor x : a\rfloor = \lfloor y/x : a\rfloor = x : a & & \lceil x : a\rceil = \lceil \dot{x} : a\rceil = x : a \\ \lfloor x : [\mathfrak{S}\,\langle\mathfrak{T})]\rfloor = \lfloor y/x : [\mathfrak{S}\,\langle\mathfrak{T})]\rfloor = x : [\lceil\mathfrak{S}\rceil\,\langle\lfloor\mathfrak{T}\rfloor)] & & \lceil x : [\mathfrak{S}\,(\mathfrak{T}\rangle]\rceil = \lceil \dot{x} : [\mathfrak{S}\,(\mathfrak{T}\rangle)]\rceil = x : [\lfloor\mathfrak{S}\rfloor\,(\lceil\mathfrak{T}\rceil\rangle)] \\ \lfloor x : [\mathfrak{S}\,\langle\mathfrak{T})\guillsinglright]\rfloor = \lfloor y/x : [\mathfrak{S}\,\langle\mathfrak{T})\guillsinglright]\rfloor = \lceil\mathfrak{S}\rceil, \lfloor\mathfrak{T}\rfloor & & \lceil x : [\mathfrak{S}\guillsinglleft(\mathfrak{T}\rangle]\rceil = \lceil \dot{x} : [\mathfrak{S}\guillsinglleft(\mathfrak{T}\rangle)]\rceil = \lfloor\mathfrak{S}\rfloor, \lceil\mathfrak{T}\rceil \end{array}$$

[7] Here we want to suggest that scroll nets provide a nested generalization of the notion of judgment. Some forms of nested judgments have been studied in Guenot's PhD thesis in the context of nested sequent systems [9, Chapters 5–6]. They are also considered in some recent categorical semantics of type theory, and in particular those in Sterling's PhD thesis [29] (see also [14] for a technical introduction to the subject).

[8] Maybe to the exception of *open deduction* and *atomic flows* [11].



(a) Non-empty outloop **(b)** Identity **(c)** Scoping

Figure 5 Examples of incorrect scroll nets

It is then possible to characterize *partial* scroll nets as those with a non-empty premiss, just as partial proofs in other formalisms are derivation trees where some leaves are not closed by a nullary rule. By analogy with terms in type theory, we will use the terminology open/closed:

▶ **Definition 10** (Openness). *A scroll net* $\mathfrak{S}$ *is said to be* closed *if* $\lceil \mathfrak{S} \rceil = \square$ *with* $\square$ *denoting the empty interface, otherwise it is* open.

5 Correctness

Incorrectness In the same way that not all proof nets are correct, Definition 6 is not restrictive enough to capture only the class of correct scroll nets. Figure 5 shows three examples of incorrect scroll nets: in Fig. 5a it is because a scroll has been collapsed despite its outloop being non-empty; in Fig. 5b it comes from the source and target of a justification having different shapes; lastly in Fig. 5c, it is caused by a justification that violates scoping by going out of a scroll that has not been collapsed yet.

Although there is probably some "geometric" criterion that could capture each of these mistakes by analyzing solely the static structure of scroll nets, in this paper we only formalize the baseline *sequential* correctness criterion: a scroll net $\mathfrak{S}$ is correct if there exists a total ordering on all its annotations (arrows in the graphical notation of Fig. 3) such that the corresponding dynamic sequence of illative transformations deduces $\lfloor \mathfrak{S} \rfloor$ from $\lceil \mathfrak{S} \rceil$.

Derivations To formalize this intuition, we define a *derivability* relation $\mathfrak{S} \rhd \mathfrak{S}'$ characterizing when $\mathfrak{S}'$ can be obtained from $\mathfrak{S}$ by recording an illative transformation, thus generalizing the visual examples of Fig. 3. Since transformations may apply deep inside scroll structures, we actually need a ternary relation $\mathcal{X} \vdash \mathfrak{S} \rhd \mathfrak{S}'$, where $\mathcal{X}$ tracks the context of nodes available for (de)iteration at the current location; the binary relation $\mathfrak{S} \rhd \mathfrak{S}'$ is then recovered as the top-level case $\mathcal{X} = \square$. Moreover, each illative transformation is applicable only in positive or only in negative locations, splitting the rules into two halves; we therefore define mutually inductively two relations $\mathcal{X} \vdash \mathfrak{S} \rhd \mathfrak{S}'$ and $\mathcal{X} \vdash \mathfrak{S} \blacktriangleright \mathfrak{S}'$, recording transformations applied respectively in positive and negative locations, by the rules of Fig. 6. We write $\frac{\mathcal{X}}{\mathcal{Y}}\mathfrak{S}$ to denote a scroll net $\mathfrak{S}$ such that $\lceil \mathfrak{S} \rceil = \mathcal{X}$ and $\lfloor \mathfrak{S} \rfloor = \mathcal{Y}$, leaving blank space when $\mathcal{X} = \square$ or $\mathcal{Y} = \square$.

Soundness We now state a series of lemmas whose proofs would ensure the adequacy of our formal definition of derivation rules. For lack of space and time, we omit such proofs.

First we want to ensure that derivation rules do not change the premiss of a scroll net:

▶ **Lemma 11** (Subject construction). *If* $\mathfrak{S} \rhd \mathfrak{T}$ *then* $\lceil \mathfrak{S} \rceil \equiv_\alpha \lceil \mathfrak{T} \rceil$.

We can then show the following results about logical soundness:

$$\textsc{Separate}\quad \frac{}{\mathcal{X} \vdash \triangleright\ x : [\langle()]}$$

$$\textsc{Reunite}\quad \frac{}{\mathcal{X} \vdash x : \left[\underline{\mathcal{Y}}_{\mathfrak{T}}\left(\overline{\mathcal{Z}}^{\mathfrak{U}}\right\rangle\right] \blacktriangleright x : \left[\underline{\mathcal{Y}}_{\mathfrak{T}}\langle\left(\overline{\mathcal{Z}}^{\mathfrak{U}}\right\rangle\right]}$$

$$\textsc{ReuniteInserted}\quad \frac{}{\mathcal{X} \vdash \dot{x} : \left[\underline{\mathcal{Y}}_{\mathfrak{T}}\left(\overline{\mathcal{Z}}^{\mathfrak{U}}\right\rangle\right] \blacktriangleright \dot{x} : \left[\underline{\mathcal{Y}}_{\mathfrak{T}}\langle\left(\overline{\mathcal{Z}}^{\mathfrak{U}}\right\rangle\right]}$$

$$\textsc{Disclose}\quad \frac{}{\mathcal{X} \vdash x : \left[\overline{\mathcal{Y}}^{\mathfrak{T}}\langle\mathfrak{U})\right] \triangleright x : \left[\overline{\mathcal{Y}}^{\mathfrak{T}}\langle\mathfrak{U})\rangle\right]}$$

$$\textsc{Enclose}\quad \frac{}{\mathcal{X} \vdash \mathfrak{U} \blacktriangleright x : [(\mathfrak{U})\rangle]}$$

$$\textsc{DiscloseIterated}\quad \frac{}{\mathcal{X} \vdash y/x : \left[\overline{\mathcal{Y}}^{\mathfrak{T}}\langle\mathfrak{U})\right] \triangleright y/x : \left[\overline{\mathcal{Y}}^{\mathfrak{T}}\langle\mathfrak{U})\rangle\right]}$$

$$\textsc{Iterate}\quad \frac{}{\mathcal{X}, y : X \vdash \triangleright\ y/x : X}$$

$$\textsc{Insert}\quad \frac{}{\mathcal{X} \vdash \blacktriangleright\ \dot{x} : X}$$

$$\textsc{DeleteAtom}\quad \frac{}{\mathcal{X} \vdash x : a \triangleright \dot{x} : a}$$

$$\textsc{DeiterateAtom}\quad \frac{}{\mathcal{X}, y : a \vdash x : a \blacktriangleright y/x : a}$$

$$\textsc{DeleteIteratedAtom}\quad \frac{}{\mathcal{X} \vdash y/x : a \triangleright y/\dot{x} : a}$$

$$\textsc{DeiterateInsertedAtom}\quad \frac{}{\mathcal{X}, y : a \vdash \dot{x} : a \blacktriangleright y/\dot{x} : a}$$

$$\textsc{DeleteScroll}\quad \frac{}{\mathcal{X} \vdash x : [\mathfrak{S}\,(\mathfrak{T})] \triangleright \dot{x} : [\mathfrak{S}\,(\mathfrak{T})]}$$

$$\textsc{DeiterateScroll}\quad \frac{}{\mathcal{X}, y : [\lfloor\mathfrak{S}\rfloor\,(\lceil\mathfrak{T}\rceil)] \vdash x : [\mathfrak{S}\,(\mathfrak{T})] \blacktriangleright y/x : [\mathfrak{S}\,(\mathfrak{T})]}$$

$$\textsc{DeleteIteratedScroll}\quad \frac{}{\mathcal{X} \vdash y/x : [\mathfrak{S}\,(\mathfrak{T})] \triangleright y/\dot{x} : [\mathfrak{S}\,(\mathfrak{T})]}$$

$$\textsc{DeiterateInsertedScroll}\quad \frac{}{\mathcal{X}, y : [\lfloor\mathfrak{S}\rfloor\,(\lceil\mathfrak{T}\rceil)] \vdash \dot{x} : [\mathfrak{S}\,(\mathfrak{T})] \blacktriangleright y/\dot{x} : [\mathfrak{S}\,(\mathfrak{T})]}$$

$$\textsc{DeleteOpened}\quad \frac{}{\mathcal{X} \vdash x : \left[\underline{\mathcal{Y}}_{\mathfrak{S}}\langle(\mathfrak{T})\right] \triangleright \dot{x} : \left[\underline{\mathcal{Y}}_{\mathfrak{S}}\langle(\mathfrak{T})\right]}$$

$$\textsc{DeiterateOpened}\quad \frac{}{\mathcal{X}, y : [\mathcal{Y}\,(\lceil\mathfrak{T}\rceil)] \vdash x : \left[\overline{\mathcal{Y}}^{\mathfrak{S}}\,(\mathfrak{T})\rangle\right] \blacktriangleright y/x : \left[\overline{\mathcal{Y}}^{\mathfrak{S}}\,(\mathfrak{T})\rangle\right]}$$

$$\textsc{Juxtaposition}+\quad \frac{\mathcal{X}, \lfloor\mathfrak{U}, \mathfrak{U}'\rfloor \vdash \mathfrak{S} \triangleright \mathfrak{T}}{\mathcal{X} \vdash \mathfrak{U}, \mathfrak{S}, \mathfrak{U}' \triangleright \mathfrak{U}, \mathfrak{T}, \mathfrak{U}'}$$

$$\textsc{Juxtaposition}-\quad \frac{\mathcal{X}, \lceil\mathfrak{U}, \mathfrak{U}'\rceil \vdash \mathfrak{S} \blacktriangleright \mathfrak{T}}{\mathcal{X} \vdash \mathfrak{U}, \mathfrak{S}, \mathfrak{U}' \blacktriangleright \mathfrak{U}, \mathfrak{T}, \mathfrak{U}'}$$

$$\textsc{Inloop}+\quad \frac{\mathcal{X}, \lceil\mathfrak{U}\rceil \vdash \mathfrak{S} \triangleright \mathfrak{T}}{\mathcal{X} \vdash [\mathfrak{U}\,\langle\mathfrak{S}\rangle] \triangleright [\mathfrak{U}\,\langle\mathfrak{T}\rangle]}$$

$$\textsc{Inloop}-\quad \frac{\mathcal{X}, \lfloor\mathfrak{U}\rfloor \vdash \mathfrak{S} \blacktriangleright \mathfrak{T}}{\mathcal{X} \vdash [\mathfrak{U}\,\langle\mathfrak{S}\rangle] \blacktriangleright [\mathfrak{U}\,\langle\mathfrak{T}\rangle]}$$

$$\textsc{Outloop}+\quad \frac{\mathcal{X} \vdash \mathfrak{S} \blacktriangleright \mathfrak{T}}{\mathcal{X} \vdash [\mathfrak{T}\,\langle\mathfrak{U}\rangle] \triangleright [\mathfrak{S}\,\langle\mathfrak{U}\rangle]}$$

$$\textsc{Outloop}-\quad \frac{\mathcal{X} \vdash \mathfrak{S} \triangleright \mathfrak{T}}{\mathcal{X} \vdash [\mathfrak{T}\,\langle\mathfrak{U}\rangle] \blacktriangleright [\mathfrak{S}\,\langle\mathfrak{U}\rangle]}$$

Figure 6 Derivation rules for scroll nets

▶ **Definition 12** (Interpretation). *The interpretation* $[\![\Phi]\!]$ *of a scroll structure* Φ *is the formula defined recursively as follows:*

$$[\![\phi_1, \ldots, \phi_n]\!] = [\![\phi_1]\!] \wedge \ldots \wedge [\![\phi_n]\!] \qquad [\![a]\!] = a \qquad [\![\,[\Phi\,(\Psi)]\,]\!] = [\![\Phi]\!] \Rightarrow [\![\Psi]\!]$$

We write $\llceil\mathfrak{S}\rceil\!\rceil$ *and* $\llfloor\mathfrak{S}\rfloor\!\rfloor$ *as shorthands for* $[\![\downharpoonleft \lceil\mathfrak{S}\rceil \downharpoonleft]\!]$ *and* $[\![\downharpoonleft \lfloor\mathfrak{S}\rfloor \downharpoonleft]\!]$.

▶ **Lemma 13** (Conclusion entailment). *If* $\mathfrak{S} \rhd \mathfrak{T}$ *then* $\llfloor\mathfrak{S}\rfloor\!\rfloor \vdash \llfloor\mathfrak{T}\rfloor\!\rfloor$.

Let $\rhd^*$ denote the reflexive and transitive closure of the relation $\rhd$.

▶ **Theorem 14** (Soundness). *If* $\mathcal{X} \rhd^* \mathfrak{S}$ *then* $[\![\downharpoonleft \mathcal{X} \downharpoonleft]\!] \vdash \llfloor\mathfrak{S}\rfloor\!\rfloor$.

▶ **Corollary 15.** *If* $\boxed{} \rhd^* \mathfrak{S}$ *then* $\vdash \llfloor\mathfrak{S}\rfloor\!\rfloor$.

Finally, the above soundness results legitimate our sequential definition of correctness:

▶ **Definition 16** (Correctness). *We say that a scroll net* $\mathfrak{S}$ *is* correct *iff* $\lceil\mathfrak{S}\rceil \rhd^* \mathfrak{S}$.

6 Computation

Detours In natural deduction, a so-called *detour* — corresponding to a β-*redex* in simply typed λ-calculus — arises when an introduction rule on some formula is followed by an elimination rule on the same formula. Similarly in scroll nets, we will call *detour* a node that is both introduced and eliminated, in the sense of being entirely scribed and then entirely erased from SA. A simple argument by combinatorial exhaustion shows that there are only 4 possible shapes of detours:
Interaction/Interaction expanded then collapsed;
Interaction/Argumentation expanded then deleted, or inserted then collapsed;
Argumentation/Interaction iterated then collapsed, or expanded then deiterated;
Argumentation/Argumentation iterated then deleted, or inserted then deiterated.
Although we have given 8 cases, they can be divided in 4 pairs that have the same shape: it is then the polarity of the detour that determines in which order the two illative transformations have been performed.

Equational theory Let $\mathfrak{T} \circ \mathfrak{S}$ denote the *sequential composition* of two scroll nets with compatible boundaries $\lfloor\mathfrak{S}\rfloor = \lceil\mathfrak{T}\rceil$, i.e. the scroll net resulting from the composition of two (arbitrarily chosen) derivations $\lceil\mathfrak{S}\rceil \rhd^* \mathfrak{S}$ and $\lfloor\mathfrak{S}\rfloor \rhd^* \mathfrak{T}$. Then the following three equations should be satisfied for any scroll net $\mathfrak{S}$:

$$\begin{aligned} \cdot\lfloor\mathfrak{S}\rfloor \circ \mathfrak{S} &= \cdot\lceil\mathfrak{S}\rceil \\ (\vec{x} : \lfloor\mathfrak{S}\rfloor, \vec{x}/\lfloor\mathfrak{S}\rfloor) \circ \mathfrak{S} &= (\mathfrak{S}, \mathfrak{S}) \circ (\vec{y} : \lceil\mathfrak{S}\rceil, \vec{y}/\lceil\mathfrak{S}\rceil) \\ [\langle(\mathfrak{S})\rangle] &= \mathfrak{S} \end{aligned}$$

In particular, the first two equations correspond to the cocommutative comonoid laws satisfied by any object in a cartesian monoidal category [1].

Simulating STLC It is possible to simulate the simply typed λ-calculus. We express this translation diagrammatically in Figure 7, awaiting future work for a more rigorous formalization of general detour-elimination rules. The translation is divided into two parts:

Static typing (Figure 7a) Each typing rule with premisses $\Gamma_i \vdash t_i : A_i$ for $1 \leq i \leq n$ and conclusion $\Gamma \vdash t : A$ is mapped to a scroll net $\mathfrak{S}$ such that $\lceil \mathfrak{S} \rceil$ (resp. $\lfloor \mathfrak{S} \rfloor$) is equal to (the translation of) Γ (resp. A), and where the inductive translations of sub-derivations appear as subnets $\mathfrak{S}_i$ with corresponding boundaries $\lceil \mathfrak{S}_i \rceil = \Gamma_i$ and $\lfloor \mathfrak{S}_i \rfloor = A_i$.

Dynamic computation (Figure 7b) Here we simulate the β-reduction rule with detour reductions. Note that this does not trigger any form of duplication or erasure, as would be the case with substitution in λ-calculus. We believe that substitution should happen when new detours in u and t generated by the application of the previous two reductions are further reduced.

7 Conclusion

In this paper we have laid out the foundations of the theory of scroll nets, focusing on its historical and conceptual genesis, its technical formalization based on an inductive syntax, and sketching its connection to categorical semantics and the simply typed λ-calculus. There is much room for improvement and development of the latter, and even of the formalism itself: we are currently investigating a simpler system that completely removes the need for interaction rules, based on a fundamental insight made by Peirce in [24] that the Sep rule can be viewed as the iteration of a single axiomatic occurrence of the empty scroll. We now outline connections with related works and directions for future work.

Intuitionistic EGs In previous work [5], we have shown how to capture precisely provability in (full) intuitionistic first-order logic inside a variant of Oostra's system of intuitionistic EGs [22] dubbed *flower calculus*. Although it enjoys a nice metaphorical notation and some useful properties in the context o f interactive theorem proving — such as *analyticity* and *invertibility* of all inference rules, it breaks the perfect duality stemming from the illative atomicity of rules found in the original approach of Peirce and Oostra, which was shown in this paper to be essential to the application of the Curry-Howard methodology to EGs.

Generalized scroll In [22], Oostra introduces a *horizontal* generalization of the scroll where it can have an arbitrary number n of inloops, subsuming seps as the case where $n = 0$. Following the classical reading of the scroll as nested negations, one naturally interprets this construct through De Morgan equivalences by taking the *disjunction* of inloops, and the aforementioned works show how one can still capture intuitionistic logic in this setting. We have observed in preliminary work that one can also consider a *vertical* generalization by allowing nested attachment of inloops to other inloops, which should enable a novel treatment of *dual-intuitionistic* and *bi-intuitionistic* logic.

Classical logic Peirce's original system of EGs for propositional logic was called Alpha, and it captured *classical* logic by adopting the classical reading of the scroll. A straightforward way to adapt scroll nets to this setting would consist in dropping the attachments of inloops to outloops (orange coloring of edges in Fig. 4b), just as Peirce ignored them. However since seps can be seen as 0-ary scrolls, we believe that a more general treatment would keep the notion of attachment, and instead characterize classical proofs as those where (de)iterations do not preserve *continuity*; that is, where an attached inloop can be duplicated into an unattached sep. Once the right framing of classical logic has been found, we believe it could provide interesting insights into the problem of finding a good notion of *proof identity* in

classical logic [30], as well as a decomposition of the computational behavior of classical systems in the Curry-Howard tradition like the $\lambda\mu$-calculus [23] and System L [21].

Other logics Peirce had also devised extensions of Alpha that capture *first-order predicate* logic (Beta) and somewhat more speculatively *modal* and *higher-order* logics (Gamma), thus providing natural venues for extensions of scroll nets to these more expressive logics. Our discovery of the computational content of EGs and its close connection to λ-calculus should enable a *type-theoretic* approach to modal and higher-order logics, hopefully shedding light on both theory and applications to programming and interactive theorem proving.

Open deduction A striking similarity between our inductive formalization of scroll nets and the *open deduction* formalism introduced in [11] is their reliance on premiss and conclusion mappings $\lceil\cdot\rceil, \lfloor\cdot\rfloor$ (written $\mathsf{pr}(\cdot), \mathsf{cn}(\cdot)$ in [11]) to identify the boundaries of proofs. Although we did not mention it in the paper, one can also compose scroll nets "by logical connective" as in open deduction: given $\mathfrak{S} : \mathcal{X} \to \mathcal{Y}$ and $\mathfrak{S}' : \mathcal{X}' \to \mathcal{Y}'$, one can take their *juxtaposition* $\mathfrak{S}, \mathfrak{S}' : \mathcal{X}, \mathcal{X}' \to \mathcal{Y}, \mathcal{Y}'$ as well as their *"scrolling"* $[\mathfrak{S}\,(\mathfrak{S}')] : [\mathcal{Y}\,(\mathcal{X}')] \to [\mathcal{X}\,(\mathcal{Y}')]$, corresponding respectively to composition by *conjunction* and *implication*. The possibility and contravariance of scrolling is due to the fact that each illative transformation performing an introduction (first row in Fig. 2) has its dual performing the converse elimination in locations of opposite polarity (second row in Fig. 2). However, we cannot formalize (dynamic) illative transformations as an open deduction calculus for two reasons:

1. the inductive syntax of scroll structures is *variadic*, whereas open deduction needs logical relations with a fixed arity;
2. the (de)iteration rules are *context-dependent*: assuming a notion of scroll structure with a hole $\hat{\Xi}$, the Iter rule could for instance be expressed as

$$\frac{\hat{\Xi}^{+}\{\boxed{}\}}{\hat{\Xi}^{+}\{\phi\}}$$

But then we need a side condition ensuring that ϕ is "available" or "visible" in $\hat{\Xi}$, as it is done with the *pollination* relation of the flower calculus [5].

Combinatorial proofs Intuitionistic combinatorial proofs have been recently introduced by Heijltjes et al. as "a concrete geometric semantics of intuitionistic logic" [12]. They also benefit from a computational interpretation [13] and have been related to game semantics [12]. However they exhibit a quite different graph-theoretic structure, with a strict separation between formulas in the arena and formulas in the game. In contrast, nodes in the scroll structure and argumentation/interaction of a scroll net are shared, leading to a more compact representation. Like scroll structures, arenas identify formulas that are equivalent modulo commutativity and associativity of conjunction. But contrary to scroll structures, they also conflate formulas equivalent modulo currying, which makes them unable to express computations under nested abstractions.

Bigraphs Scroll nets are closely related to the notion of *bigraph* introduced by Milner as a model of mobile interaction [19]. A bigraph consists of two independent structures: a *topograph* or *place graph* encoding spatial information as a forest, and a *monograph* or *link graph* encoding the connectivity of nodes as a hypergraph sharing the same vertices as the topograph. They have been generalized to *directed bigraphs* in [8] by orienting the monograph's edges to encode resource dependencies or information flow. Scroll structures

can be seen as topographs, while the arrow notation for (static) illative transformations seems to be an instance of monograph in directed bigraphs. It is remarkable that scroll nets are an instance of a very recent model of computation, while being based entirely on the principles of EGs that predate both proof theory and the advent of computer science. It would be interesting to explore the possibility of formalizing illative transformations and detour reduction rules as a *bigraphical reactive system*, possibly in a categorical setting such as adhesive categories [16].

References

1. Comonoid in nLab. URL: https://ncatlab.org/nlab/show/comonoid#InACartesianMonoidalCategory.
2. M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Lévy. Explicit substitutions. *Journal of Functional Programming*, 1(4):375–416, October 1991. doi:10.1017/S0956796800000186.
3. Filippo Bonchi, Alessandro Di Giorgio, Nathan Haydon, and Pawel Sobocinski. Diagrammatic Algebra of First Order Logic, January 2024. arXiv:2401.07055 [cs, math]. URL: http://arxiv.org/abs/2401.07055, doi:10.48550/arXiv.2401.07055.
4. Geraldine Brady and Todd H. Trimble. A categorical interpretation of C.S. Peirce's propositional logic Alpha. *Journal of Pure and Applied Algebra*, 149(3):213–239, June 2000. URL: https://linkinghub.elsevier.com/retrieve/pii/S0022404998001790, doi:10.1016/S0022-4049(98)00179-0.
5. Pablo Donato. The Flower Calculus. In Jakob Rehof, editor, *9th International Conference on Formal Structures for Computation and Deduction (FSCD 2024)*, volume 299 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 5:1–5:24, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.FSCD.2024.5.
6. Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1 – 101, 1987. doi:10.1016/0304-3975(87)90045-4.
7. Jean-Yves Girard. *Proofs and Types*. Number 7 in Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, Cambridge [England] ; New York, 1989.
8. Davide Grohmann and Marino Miculan. Directed Bigraphs. *Electronic Notes in Theoretical Computer Science*, 173:121–137, April 2007. doi:10.1016/j.entcs.2007.02.031.
9. Nicolas Guenot. *Nested Deduction in Logical Foundations for Computation*. PhD thesis, Ecole Polytechnique X, April 2013. URL: https://pastel.archives-ouvertes.fr/pastel-00929908.
10. Alessio Guglielmi. A calculus of order and interaction. Technical report, Technische Universität Dresden, 1999. URL: https://www.researchgate.net/publication/2807151_A_Calculus_of_Order_and_Interaction.
11. Alessio Guglielmi, Tom Gundersen, and Michel Parigot. A Proof Calculus Which Reduces Syntactic Bureaucracy. In Christopher Lynch, editor, *Proceedings of the 21st International Conference on Rewriting Techniques and Applications*, volume 6 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 135–150, Dagstuhl, Germany, 2010. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.RTA.2010.135.
12. Willem B. Heijltjes, Dominic J. D. Hughes, and Lutz StraBburger. Intuitionistic proofs without syntax. In *2019 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 1–13, Vancouver, BC, Canada, June 2019. IEEE. URL: https://ieeexplore.ieee.org/document/8785827/, doi:10.1109/LICS.2019.8785827.
13. Willem B. Heijltjes, Dominic J. D. Hughes, and Lutz Straßburger. Normalization Without Syntax. *LIPIcs, Volume 228, FSCD 2022*, 228:19:1–19:19, 2022. doi:10.4230/LIPICS.FSCD.2022.19.
14. Xu Huang. Synthetic tait computability the hard way, 2023. URL: http://arxiv.org/abs/2310.02051, arXiv:2310.02051.

15 Dominic Hughes. Proofs without syntax. *Annals of Mathematics*, 164(3):1065–1076, 11 2006. doi:10.4007/annals.2006.164.1065.

16 Stephen Lack and Paweł Sobociński. Adhesive and quasiadhesive categories. *RAIRO - Theoretical Informatics and Applications*, 39(3):511–545, July 2005. doi:10.1051/ita:2005028.

17 Minghui Ma and Ahti-Veikko Pietarinen. A graphical deep inference system for intuitionistic logic. *Logique et Analyse*, 245:73–114, January 2019. URL: http://www.scopus.com/inward/record.url?scp=85066258215&partnerID=8YFLogxK, doi:10.2143/LEA.245.0.3285706.

18 Paul-André Melliès. Functorial boxes in string diagrams. In Zoltán Ésik, editor, *Computer Science Logic*, pages 1–30, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg. URL: https://www.irif.fr/~mellies/papers/Mellies06csl.pdf.

19 Robin Milner. Bigraphical Reactive Systems. In *Proceedings of the 12th International Conference on Concurrency Theory*, CONCUR '01, pages 16–35, Berlin, Heidelberg, August 2001. Springer-Verlag.

20 Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, pages 625–635, Cham, 2021. Springer International Publishing.

21 Guillaume Munch-Maccagnoni. Focalisation and Classical Realisability. In Erich Grädel and Reinhard Kahle, editors, *Computer Science Logic*, volume 5771, pages 409–423. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009. doi:10.1007/978-3-642-04027-6_30.

22 Arnold Oostra. *Los gráficos Alfa de Peirce aplicados a la lógica intuicionista*. Number 2 in Cuadernos de Sistemática Peirceana. Centro de Sistemática Peirceana, 2010. URL: https://www.academia.edu/36611719/Cuadernos_de_Sistem%C3%A1tica_Peirceana_1.

23 Michel Parigot. $\Lambda\mu$-Calculus: An algorithmic interpretation of classical natural deduction. In Andrei Voronkov, editor, *Logic Programming and Automated Reasoning*, pages 190–201, Berlin, Heidelberg, 1992. Springer. doi:10.1007/BFb0013061.

24 Charles Sanders Peirce. Prolegomena to an Apology for Pragmaticism. *The Monist*, 16(4):492–546, 1906. Publisher: Oxford University Press. URL: https://www.jstor.org/stable/27899680.

25 Ahti-Veikko Pietarinen. Peirce's game-theoretic ideas in logic. *Semiotica*, 2003:33–47, January 2003. URL: https://www.researchgate.net/publication/249934518_Peirce%27s_game-theoretic_ideas_in_logic, doi:10.1515/semi.2003.030.

26 Don D. Roberts. *The Existential Graphs of Charles S. Peirce*. De Gruyter Mouton, Berlin, Boston, 1973. doi:10.1515/9783110226225.

27 Don D. Roberts. The existential graphs. *Computers & Mathematics with Applications*, 23(6-9):639–663, March 1992. URL: https://linkinghub.elsevier.com/retrieve/pii/0898122192901274, doi:10.1016/0898-1221(92)90127-4.

28 Sun-Joo Shin. *The Iconic Logic of Peirce's Graphs*. The MIT Press, 05 2002. doi:10.7551/mitpress/3633.001.0001.

29 Jonathan Sterling. First Steps in Synthetic Tait Computability: The Objective Metatheory of Cubical Type Theory. 4 2022. URL: https://kilthub.cmu.edu/articles/thesis/First_Steps_in_Synthetic_Tait_Computability_The_Objective_Metatheory_of_Cubical_Type_Theory/19632681, doi:10.1184/R1/19632681.v1.

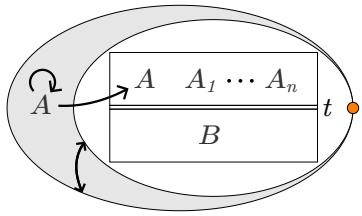
30 Lutz Straßburger. The problem of proof identity, and why computer scientists should care about hilbert's 24th problem. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 377(2140):20180038, 2019. doi:10.1098/rsta.2018.0038.

31 The Rocq Development Team. The rocq prover. Zenodo, April 2025. doi:10.5281/zenodo.15149629.

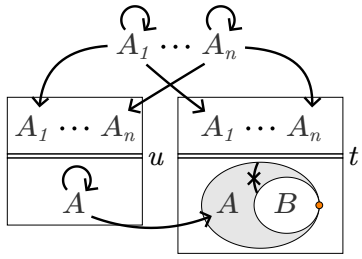
Let $\Gamma = A_1, \ldots, A_n$.

$$\frac{}{\Gamma, x : A \vdash x : A}\,\mathsf{var} \quad \mapsto$$

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x.t : A \to B}\,\mathsf{abs} \quad \mapsto$$

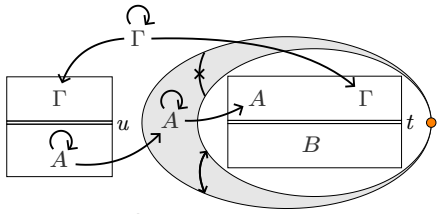


$$\frac{\Gamma \vdash t : A \to B \quad \Gamma \vdash u : A}{\Gamma \vdash (t)u : B}\,\mathsf{app} \quad \mapsto$$

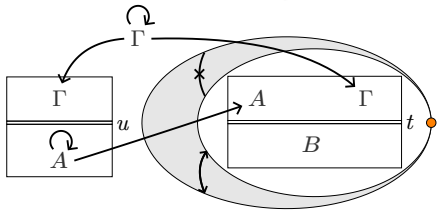


(a) Typing rules

$$\frac{\dfrac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x.t : A \to B}\,\mathsf{abs} \quad \Gamma \vdash u : A}{\Gamma \vdash (\lambda x.t)u : B}\,\mathsf{app}$$

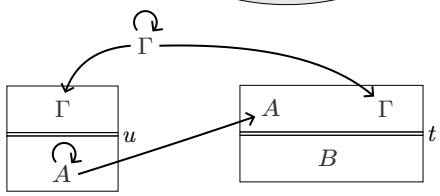


$\rightsquigarrow_{\mathsf{aa}}$



$\rightsquigarrow_{\beta}$ $\Gamma \vdash t\{u/x\} : B$

$\rightsquigarrow_{\mathsf{ii}}$



(b) β-reduction

Figure 7 Simulation of simply typed λ-calculus